

A Formalisation of the Futurama Theorem in Lean

CHENG Yaoyu, A0304680J; HUANG Tuosen, A0297004H

Project 10

April 2026

1 Problem and Motivation

1.1 Background and the Scrambling Setup

This project studies the constructive restoration theorem behind *The Prisoner of Benda*, a result originally devised to resolve a plot hole in an episode of the animated series *Futurama*. The theorem was later abstracted and given a rigorous mathematical treatment by Evans, Huang, and Nguyen [1].

The operational setup is as follows: suppose n individuals have had their bodies scrambled by a mind-swapping machine. The machine operates strictly by swapping the minds of two distinct individuals, but with a critical physical constraint: each pair of bodies can only be swapped at most once. The resulting scrambled state can be represented as a permutation $\sigma \in S_n$. Now the question is: With this strict no-repeat rule, can we always restore everyone to their original mind-body state? The answer is yes, so long as we introduce two “fresh” helpers, denoted as x and y , who took no part in the original scramble. (Following our Lean formalisation, we write σ for the scrambled permutation; the reference paper [1] uses P for this role.)

1.2 The Futurama Theorem and Constructive Repair

Below we state the rigorous mathematical result that guarantees this restoration:

Futurama Theorem. *Let σ be a product of distinct transpositions in S_n , that is, a permutation arising from a sequence of pairwise-distinct mind switches. There exists a product τ of distinct transpositions in S_{n+2} , each containing an entry in $\{x, y\}$, such that τ undoes σ ; in particular, no transposition pair from the original scramble is reused.*

The proof employs a constructive, cycle-by-cycle algorithm. We begin by decomposing the initial scramble $\sigma = C_1 \cdots C_r$ into a product of disjoint cycles. For each cycle $C_i = (a_1 a_2 \cdots a_k)$, we construct a dedicated repair block σ_i using the helpers x and y , such that:

$$\sigma_i \cdot C_i = (x y).$$

Each cycle repair introduces a residual transposition $(x y)$, so the full repair process leaves a residual state dependent on the parity of r . After repairing all r cycles, the helpers are left in the state $(x y)^r$. If r is odd, this residual state is easily resolved with a final swap of the two helpers x and y . The core equation for the entire restoration process is:

$$\tau \cdot \sigma = \text{id},$$

where τ denotes the concatenation of all individual repair blocks, followed by the optional parity cleanup step. As a concrete example, consider the 3-cycle $(a b c)$. In our formalisation, the repair block executes five consecutive swaps: (x, a) , (y, b) , (y, c) , (x, b) , (y, a) ; the reference paper [1] uses a different but equivalent ordering. In both cases, direct computation confirms that $\sigma_{(abc)} \cdot (a b c) = (x y)$.

1.3 Formalisation Goals and Mechanisation Challenges

Our goal is to mechanise this construction in Lean. Although the informal proof is concise, its operational logic is more subtle than it first appears: one must pin down precise definitions for configurations, swap scripts, permutation composition, and cycle representations before any reasoning can begin. These definitions, in turn, expose assumptions that the paper leaves implicit. For example, the paper silently takes for granted that the helpers x and y are unaffected by the scrambled permutation. In Lean, this must be proved explicitly, and doing so cleanly requires a type-level separation between original cast members and helpers. Bridging such gaps between paper proofs and fully verified formal systems is the central challenge of the project.

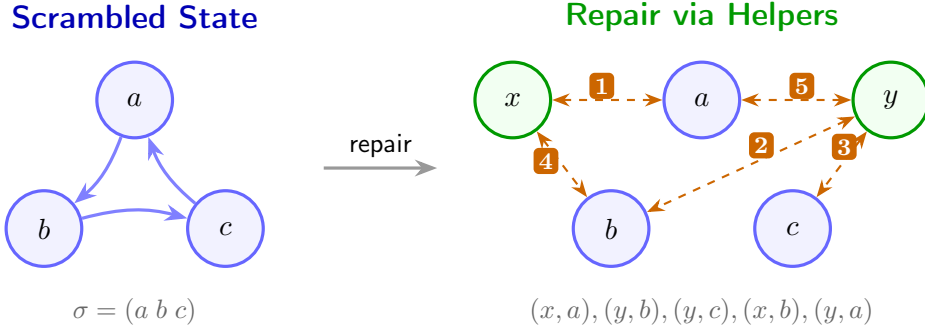


Figure 1: Single-cycle repair of $(a\ b\ c)$

2 Relation to the Course and Methodology

All our work is built in Lean [2]. Swap configurations, repair scripts, and correctness theorems are all represented in the logic of the proof assistant, so the work uses the central skills of the course [3]: first formalize the definitions, decompose the proof into auxiliary lemmas, and then build reusable proof infrastructure around a nontrivial theorem.

Although our development is not a Veil or Velvet project, it follows the same operational discipline those frameworks use. We model swaps as explicit state transformations, give repair scripts an executable semantics, and prove global properties by composing local transition lemmas and invariants.

On the Lean side, the formalisation makes heavy use of Mathlib’s `Equiv.Perm` library for permutations. The state space is an inductive type `Body α` with three constructors—`orig a` for each member a of the original cast, plus `x` and `y` for the two helpers. Cycles are represented as a structure `Cycle α` carrying at least two distinct elements and a proof of no duplicates. Repair scripts are values of type `List (Body α × Body α)`, executed left-to-right by a function `runScript` that composes transpositions by right-multiplication. An arbitrary permutation σ on the original cast is lifted to `Body α` via a helper-fixing embedding `liftPerm`, and the intended top-level correctness theorem is `runScript (undoScriptOfPerm σ) * liftPerm σ = 1`. Getting all of this to typecheck requires sustained interaction with Lean’s type system, Mathlib’s permutation API, and the decidable-equality machinery for `Body α`.

3 Feasibility and Current Progress

We are well on track to meet the April 27, 2026 deadline. The foundational mechanisation infrastructure is already up and running in `Lean/Project/TestProject.lean`. In concrete terms, we have defined the core mathematical objects (configurations, swap scripts, and cycles) and proved the supporting lemmas needed to verify basic reasoning patterns. Although the overarching constructive theorem is not yet finished, having these building blocks in place gives us strong confidence that the remaining formalisation is entirely tractable.

The main technical hurdle going forward lies in the interaction between cycle decomposition and the “freshness” constraints on our helpers. More specifically, stitching the per-cycle repairs together requires a proof that the helper swap $(x\ y)$ commutes with any sequence of original cast members. On paper this is immediate, because the two sets are completely disjoint; in Lean, however, this disjointness must be established explicitly, which tends to be verbose.

Our current progress and future plans are summarised in the table below.

Timeline	Milestone	Status & Objectives
Up to Present	Infrastructure & Core Lemmas	Completed: Defined configurations, basic swap mechanics, and single-cycle properties in Lean.
Week 1 (Apr 10 – 15)	Theorem Assembly & Constraints	In Progress: Composing per-cycle arguments into the full theorem; resolving tedious disjointness proofs.
Week 2 (Apr 16 – 21)	Paper Correspondence & Draft	Planned: Writing the explanatory report; explicitly mapping our Lean definitions to the informal reference paper.
Week 3 (Apr 22 – 27)	Final Polish & Stretch Goals	Planned: Code cleanup, report formatting.

Table 1: Timeline for reference

4 Deliverables: Bare Minimum and Stretch Goals

Bare formalisation minimum. The project’s minimum success bar is a well-engineered Lean mechanisation of the constructive Futurama theorem, valid for arbitrary finite scrambled configurations and relying on two fresh helpers. The main points are as follows:

- to give executable definitions for swap scripts and repair procedures;
- to establish a machine-checked correctness theorem $\tau \cdot \sigma = \text{id}$ on $\text{Body } \alpha$, where σ is the scrambled permutation and τ the constructed repair;
- to verify that each step in the repair script involves a helper, and that no transposition pair is reused;
- to provide a short written explanation covering our key modeling choices, including how we represent configurations, helpers, cycles, and script execution.

Possible stretch goals. If the core theorem is completed early enough, we would like to extend the project in the direction of the later sections of [1]. The most natural stretch goal is a mechanised presentation of the optimality theorem showing the $n + r + 2$ bound for the improved repair construction. A second stretch goal is a clearer formal alignment with the later sections of the paper, including selected special-case results when time permits. These extensions are valuable, but they are explicitly secondary to the constructive theorem itself.

The constructive restoration theorem is the core deliverable; the optimality layer and broader paper coverage are welcome extensions but not required for success.

References

- [1] R. Evans, L. Huang, and T. Nguyen. Keeler’s Theorem and Products of Distinct Transpositions. *Amer. Math. Monthly*, 121(2):136–144, 2014.
- [2] L. de Moura and S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In *CADE 28*, pages 625–635, 2021.
- [3] I. Sergey. CS5232: Formal Specification and Design Techniques. Course materials, NUS, 2026.